🕑 Kubernetes Lab Challenge: CronJob with Retry Logic

🎯 Objective

Create a **Kubernetes CronJob** in EKS that runs periodically, simulates failures, and implements retry logic using native mechanisms.

🧭 Scenario

You need to run a daily batch job that processes files. However, failures are expected occasionally (e.g., API rate limits, flaky services). Your goal is to:

- Create a **CronJob** that runs every minute
- Simulate failure in the command logic
- Configure **retry strategy** to tolerate and recover

🧰 Tools & Concepts

- AWS EKS
- Kubernetes CronJob
- Pod restart policy
- Backoff limit & activeDeadlineSeconds

🔧 Tasks

🟢 Step 1: Create a Simple Failing Job Script

Write a script (e.g., Bash, Python) that randomly fails with exit code 1:
```bash
#!/bin/sh
echo "Running job at $(date)"
[ $((RANDOM % 2)) -eq 0 ] && echo "Failing..." && exit 1
echo "Success!"
```

Make sure the image you use includes `sh` or `bash`.

🟢 Step 2: Define CronJob Resource

Create a CronJob YAML that:

- Runs every minute (`*/1 * * * *`)
- Uses `restartPolicy: Never`
- Has `backoffLimit: 3` and `activeDeadlineSeconds: 100`

📘 Reference:
- https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/

🟢 Step 3: Apply and Observe

```bash
kubectl apply -f my-cronjob.yaml
kubectl get cronjob
kubectl get jobs --watch
kubectl logs job/<job-name>
```

Observe successful and failed runs.

🟢 Step 4: Enhance with Retry Patterns

Try alternatives:
- Implement retries inside the job script
- Use `restartPolicy: OnFailure` for Jobs

💡 You can also wrap logic using tools like `retry` or custom loop in shell script.

✅ Completion Criteria

- CronJob runs every minute
- Retries on failure using `backoffLimit`
- Logs show intermittent failures handled gracefully

🚀 Bonus Challenges

- Create a success alert via `kubectl logs | grep Success`
- Add a Job that posts a message to Slack on success
- Write job logs to a persistent volume (e.g., EFS)

📝 Deliverables

Be ready to demonstrate:
- CronJob spec with retry config
- Simulated failures and retries
- Logs showing graceful handling